Answers you can check:

a glass-box process simulator for engineering students working with AI agents

Vítor Geraldes[1] Pedro Mendes[2]
Miguel Rodrigues[3]

TODO (Vítor): affiliations 1–3, ORCIDs and the corresponding author.

White paper · development line `Choupo-dev` · DRAFT of 21 October 2026

Licence. The prose and figures of this white paper are licensed under the Creative Commons Attribution-ShareAlike 4.0 International licence (CC BY-SA 4.0). Code excerpts and Choupo case files quoted in it remain under the GNU General Public License, version 3 or later (GPL-3.0-or-later), the licence of the software they come from. *Choupo* is a registered trademark of TalentGround Lda.; the trademark is not licensed by either licence.

Abstract

A language model can now write a plausible five-hundred-page process design in minutes. Plausible is not verified, and a student handed such a design has no way to tell which of its numbers were computed and which were remembered. We argue that the value of a process simulator in teaching is moving from *producing* a design to *proving* one, and that an AI agent should not compute engineering answers from memory but call a deterministic, open and auditable tool and show what it did. We describe Choupo, an open-source (GPL-3.0-or-later), dependency-free C++17 process simulator built around that idea: it refuses by name rather than default in silence, closes mass, atoms and energy on every converged steady run by default, carries a source and a licence for every curated value, and keeps a small named subset of cases anchored to published measurements. Its development is organised so that each class of error, once found, gets a gate that makes it unable to leave in silence. We compare it on a published sample case with an open-source simulator, describe how a student works with an agent around it, and state plainly what it does not do.

DRAFT

1 The problem: plausible is not verified

An engineering student today can ask an AI agent for the design of a distillation train, an ammonia loop or a crystalliser, and receive within minutes a document that has every outward mark of competence: a flowsheet, stream tables, duties, equipment sizes, a cost estimate, a page of assumptions. Some of its numbers will be right. Some will be the kind of error a reviewer spends a career learning to spot: an energy balance that does not close, a vapour pressure extrapolated eighty kelvin past its fit, a heat of reaction with the wrong reference state, a column fed a liquid that is in fact two-phase, a cost on a volume computed in kilomoles. None of these errors announces itself. A language model produces text that is statistically likely; an energy balance is not a matter of likelihood.

For a course this changes the question. Producing a design is becoming cheap. What remains expensive, and what a graduate will be paid for, is knowing *which numbers in a design can be trusted, and how one would know.* A design that cannot be checked is not worth much more than the time it took to generate, however long it is.

The usual answer is “run it through a simulator”. But most simulators a student meets were built for a different reader: an expert who already knows what the program does when a

parameter is missing, a model is used out of range, or an iteration stops short. The student sees converged numbers. Whether a missing interaction parameter was silently set to zero, whether a recycle converged or merely stopped, whether the enthalpy reference of the reactor agrees with that of the stream table, is not on the screen. Handing an agent such a tool moves the opacity from the agent to the tool; it does not remove it.

This paper describes a tool designed for the other reader: one who must be able to check every answer, and who increasingly works through an agent.

2 The thesis: the agent calls the tool, the tool shows its work

Our thesis has two halves.

First, an agent should not compute engineering answers from memory. It should call a deterministic, open and auditable program and report what that program did: which models it assembled, which data it read and from where, which iterations it took, which balances closed and which did not. The agent's strengths — reading documentation, drafting case files, fitting a parameter, explaining an equation — are real, and they are best used to *prepare* a computation that something else performs and checks.

Second, the value of a design moves from who wrote it to what proves it right. If an agent writes the case files and the simulator solves them, the student's contribution is no longer typing; it is judgement. That judgement needs evidence on the screen: a refusal that names what is missing, a conservation report that says how far a plant is from closing, a caveat that says which property was extrapolated. A simulator built for this reader must therefore be *glass-box* in a strong sense: not only open source, but explicit at run time about every assumption it makes.

Choupo is itself developed under this thesis. Substantial parts of its code, documentation and tutorial corpus were drafted by AI agents working under written briefs, reviewed and curated by the human maintainers; a change is integrated only after the validation its reach requires has passed, and the engine's checks, not the author's confidence, decide whether an answer has moved. The rest of this paper argues that this is a workable arrangement only because the checks were designed first.

DRAFT

3 How Choupo works

Choupo is a steady-state and dynamic process simulator written in C++17 with no external libraries: the Newton solvers, the linear algebra, the integrators, the optimisers and the dictionary parser are all hand-written, readable, and part of the source a student can open.

3.1 Three layers

The engine has three layers (Figure 1). The *simulator core* turns one flowsheet into one solution — streams, unit results and key performance indicators — as a pure function of its inputs. An optional *outer driver* calls that function repeatedly: parameter sweeps, design specifications, optimisation, parameter estimation. An optional *post-processor* chain augments a converged solution with equipment sizing, costing and reports. Five binaries partition the problem classes: `choupoSolve` (steady state, $F(x) = 0$, with recycles solved by Newton on the tear streams), `choupoBatch` (batch campaigns driven by a recipe), `choupoSemiContinuous` (a time-integrated flowsheet without control loops: start-up, fed-batch, feed-and-bleed), `choupoCtrl` (the same integration with a control layer) and `choupoProps` (a property workbench: evaluations, consistency tests, parameter fitting). Numerical strategies coexist inside each binary and are selected in the case files; there is no binary per algorithm.

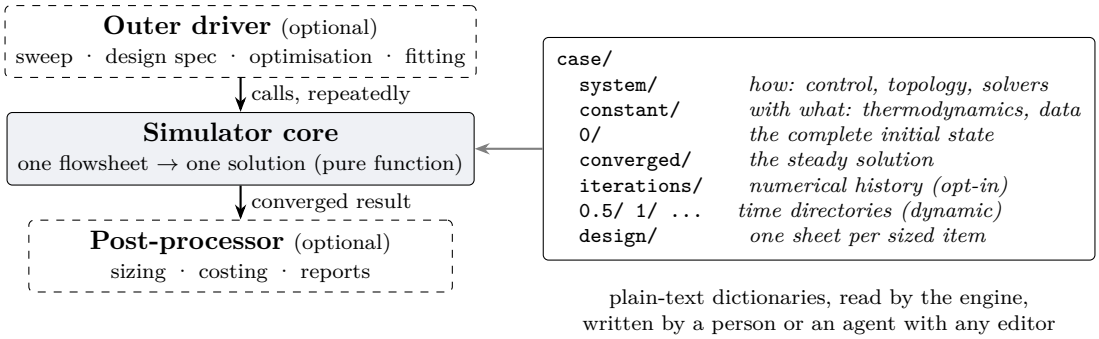


Figure 1: The three layers of the engine and the case directory they read and write. Dashed layers are optional. The browser front-end runs the same compiled engine and draws its outputs; it computes no physics of its own.

3.2 The case is a directory of plain-text dictionaries

A case is a directory. `system/` says *how* (control, flowsheet topology, solver options, the outer driver, post-processing), `constant/` says *with what* (the thermophysical system, reactions, case-local data), and `0/` holds the complete initial state, one file per stream. The flowsheet dictionary carries topology only; a stream's values live in its own file, and a missing or orphan stream file is a fatal error. Listing 1 is the whole topology and the feed of the first tutorial, an isothermal flash. Every dimensional scalar carries its unit and is converted to SI and dimension-checked on reading.

Listing 1: The first tutorial's topology, `system/flowsheetDict`, and its feed, `0/feed` (case files, GPL-3.0-or-later; banners omitted).

```
units
(
    {
        name        flash01;
        type        isothermalFlash;
        model       rachfordRice;      // the machine, NAMED (type -> model -> operation)
        in          feed;
        outputs     ( liquid  vapor );
        operation   { T 370.0 K;  P 1.0 bar; }
    }
);
// ************************************************************************* 0/feed
componentMolarFlows { benzene 40 kmol/h;  toluene 60 kmol/h; }
T               370 K;
P               100000 Pa;
```

Explicit factories. Every extensible base class — unit operations, activity models, equations of state, outer drivers, post-processors and the rest — has a factory, and every built-in type is registered by one line inside a `registerBuiltins()` function that `main()` calls explicitly. There are no self-registration macros and no static initialisers, so "where does this model come from?" is answered by reading one function.

State directories are restartable snapshots. The steady solution is written to `converged/` in the same layout as `0/`, and only when the run converged: the directory's name is a contract. Each unit's internal state (a column's stage profile, a reactor's axial profile, a crystal size distribution) is written beside the streams under `internalStates/`, so a converged directory renamed `0/` restarts the case. The time-integrated binaries write a time directory at every write interval in the same layout. `iterations/` keeps the numerical history when asked for, never physical time.

DRAFT

A plant and its sectors. A large flowsheet may be organised as a plant and its sectors, each sector a folder; the engine flattens the tree to one flat problem with qualified names, and a sector may carry its own thermodynamic model. A stream's file lives at the lowest level of the case whose subtree contains both of its ends.

The browser runs the same engine. A web front-end runs the same C++ compiled to WebAssembly. It is deliberately a *runner and visualiser*, not an editor: it loads the dictionaries, draws the flowsheet, runs the solver, streams the log and plots the results, and every balance it shows is one the engine published.

The run log shows the computation. At the default verbosity the log prints each unit as it is solved, the model it uses and where its thermodynamics came from, every Newton iteration, and every extrapolation at the place it happens. It ends with two blocks: whether the problem solved is the problem posed, and every assumption and caveat raised during the run, repeated because a warning a thousand lines above the answer has been delivered but not received. Listing 2 is an excerpt from the flash of Listing 1; note that it says, unprompted, that benzene's vapour pressure was evaluated outside the temperature range of its correlation.

TODO (authors): Regenerate Listing 2 from a build of the release this paper will cite (the excerpt was taken from a development build, commit `4185a97a9`).

Listing 2: Excerpt of the run log of the flash (`verbosity 3`); elided lines marked `[...]`.

```
>>>  Unit [0]:  flash01   (type = isothermalFlash)
>>>  thermo: inherited (global package)
  [flash] model rachfordRice  (declared)
[psat] component 'benzene': Antoine evaluated at T = 370 K, OUTSIDE its declared Trange (287.7 354.07).
K-values  (γ = ideal,  φ = idealGas,  evaluated at x = z initially):
  benzene       Psat = 1.6514 bar    K = 1.6514
  toluene       Psat = 0.6735 bar    K = 0.6735
  [outer 0] Rachford-Rice Newton (inner):
    it          V           g(V)          dg/dV          ΔV
    0     0.50000000   -3.75806e-02   -1.87920e-01   -1.99981e-01
    1     0.30001863   7.81978e-04    -1.97366e-01   3.96206e-03
    2     0.30398070   5.66359e-07    -1.97081e-01   2.87374e-06
    3     0.30398357   2.94959e-13    -1.97081e-01   0.00000e+00
[...]
  [report] massBalance       (global closure 100.000 %)
  [report] globalEnergyBoundary  (residual -0.0000 kW; NO EXCHANGED-ENERGY SCALE [...])
  [report] elementBalance    (worst element closure off by 0.0000 %)
[...]
PROBLEM SOLVED == PROBLEM POSED.  No divergence declared or imposed.
ASSUMPTIONS AND CAVEATS -- what this run announced along the way
  EXTRAPOLATIONS AND VALIDITY  (3)
    - component 'benzene': vapour pressure evaluated at T = 370 K, OUTSIDE its declared Trange [...]
```

What Choupo takes from OpenFOAM, and where it departs. Choupo's case organisation is inspired by OpenFOAM [1], an open-source (GPL) C++ toolbox for computational fluid dynamics whose transparent, plain-text dictionaries we regard as one of the most elegant designs in scientific software. From it Choupo takes: hierarchical plain-text dictionaries; the case directory as the unit of work, with `system/`, `constant/`, `0/` and state or time directories; one application per problem class; utilities as small programs in `bin/`; and a release model in which the default branch is the development line and each release is an immutable versioned tag. Where an analogy is used it is stated with its limits: Choupo's normalised residual follows OpenFOAM's normalisation factor with equations in place of cells, and its source states three reasons why it is not the same theorem. It departs deliberately in three places. Model selection uses the explicit `registerBuiltins()` factories above instead of run-time-selection macros; the domain is process simulation — streams, unit operations, phase equilibria — not continuum mechanics; and there are no external dependencies at all. **Choupo is an independent project. It is not**

DRAFT

part of, affiliated with, endorsed by, or sponsored by the OpenFOAM Foundation or OpenCFD Ltd, and no OpenFOAM source code was adapted or reused in it.

4 What makes an answer verifiable

"Open source" is necessary and far from sufficient. A program whose source can be read but whose run does not say what it assumed is checkable only by someone prepared to read the source for every run. Six properties make a Choupo answer checkable by a student, and by the agent working with one.

Refusal by name instead of a silent default. When a case is missing something a model needs, or declares a word the engine does not know, the run stops and says what is missing and what would fix it, naming the valid alternatives and, where one is close, the nearest. A model name with a typing slip does not fall through to a default model; a binary pair used outside the temperature span of its evidence is announced; an approximation the author did not authorise is refused, and one the author did authorise is printed above the results as a divergence between the problem posed and the problem solved. The same rule binds the solver's own aids: an initial guess, a tear estimate or a bound is declared in the case, and the solver reports aloud when it uses one.

Conservation, owned by the engine and on by default. Every converged steady run writes three balances — total mass, atoms per element and energy — whether or not the case asked for them, because conservation is the curriculum. A case may opt out explicitly; it cannot opt out by forgetting. Where an energy balance cannot be priced (a species without a formation enthalpy), the default report says so and names the missing datum rather than printing a zero. The browser draws these balances; it never recomputes them. These balances matter because a regression test cannot see what they see. A golden-master test pins what a run prints; a plant that loses two percent of its energy, stably, passes its golden for ever. Only a gate that reads the balance and refuses can catch it (Section 5).

A source and a licence for every value. The curated catalogue cites the primary source per value, never the aggregator's arrangement; each record says where its numbers came from and how they were obtained (measured, regressed, estimated), and a record that has not been reviewed, or comes from a private local tier, is announced as such when a run reads it. A dataset with no licence statement, or a NonCommercial one, is kept out of the public tree, and a pair table does not acquire its courier's licence by being redistributed inside another package. The engine refuses to write into the curated catalogue: new data enters it by a human act of curation.

A named validation subset. Most of Choupo's regression corpus compares each case with an answer an earlier Choupo run recorded. A pass proves the answer has not moved; it says nothing about whether it was ever right. The project therefore separates *verification* from *validation* [2] and keeps a small, named subset of cases anchored to independent published results: the Williams–Otto reactor and its published steady state [3], the IAPWS-IF97 verification tables [4], the Hamer–Wu activity coefficients of uni-univalent salts [5], a measured calcite pH [6], and others. Where an anchor is the model's own training data the case says so and is not counted as validation. The subset is small, and Section 9 says so.

A check that cannot run must not pass. A gate whose input has disappeared, or whose tool is not installed, or whose harness died, must refuse, never report success; and a tool that could not look must not report what it did not see. This sounds obvious and is the property most easily lost, because a permanently green check looks exactly like a healthy one. Each gate

DRAFT

Table 1: Ten classes of error and the gate that keeps each from leaving a run in silence. Each gate's claim is the line its own output marks as one (`generated/gateManifest.json`); each record, under `docs/design/`, tells how the class was found.

The class this gate removes	Gate	Record
A plant that does not conserve mass, read from the engine's own report on every steady case	`check_mass_closure`	`a-dryer-that-ended-wetter-than-it-started.md`
A plant whose atoms do not close, element by element	`check_element_balance`, `check_default_reports`	`what-a-passing-suite-does-not-say.md`
A first law that does not close, or an open one that grows	`check_energy_closure` (ratchet)	`what-a-passing-suite-does-not-say.md`
A unit computing with a state its streams do not carry	`check_feed_thermal_state`	`the-state-a-unit-computes-with.md`
A declared word the engine does not know, silently taken as the default	`check_model_default_registered`	`three-silences-at-exit-zero.md`
An approximation nobody authorised	`check_problem_divergence`	`problem-divergence-contract.md`
An equipment size that cannot be recomputed from the declaration	`check_design_sheet`	`a-basis-nobody-stated-and-a-volume-in-kilomoles.md`
A value whose licence does not allow redistribution	`check_source_licence`	`the-licence-of-a-sigma-profile.md`
A write into the curated catalogue by a computation	`check_standards_write_guard`	`a-rule-enforced-in-one-place.md`
A check that passes because it could not run	`runtests_verdict.py`, `check_gate_selftest`	`destructive-gate-contamination.md`

is verified by sabotage: the defect it exists to catch is introduced on purpose and the gate is observed to fail.

A re-recorded answer is a claim. When an engine change moves a recorded answer, the moved rows are listed, each with the physical or numerical reason it moved, and shown to the maintainer before anything is re-recorded. A row that moves for a reason nobody can state is treated as a regression until proven otherwise.

5 How a class of error becomes unable to leave in silence

Every simulator has errors; this one has had its share, and some reached users before the check that now catches them existed. What distinguishes a verifiable tool is not the absence of errors but what its architecture does with them. In Choupo the rule is: *when an error escapes, it becomes a gate* — an executable check that rules out not the one instance found but the whole class it belongs to, wired into the regression suite, verified by sabotage, and documented in a design record that tells how the class was found. The generated gate manifest lists 239 gates, each with the claim its own output makes; Table 1 shows ten classes and the gate that removes each.

Four of these deserve a sentence more, because each illustrates a different way a class is closed.

Conservation, measured by the engine, on every case. The mass gate reads the engine's own mass-balance report for every steady case in the corpus and refuses a plant that misses closure by more than half a percent; it never recomputes the balance itself, so the gate and the student read the same number. The class it removes was found when a spray dryer's moisture

DRAFT

model produced more water than its feed carried, while the case's golden master passed: nothing had been looking at closure.

The first law, with a ratchet. The energy gate reads each plant's boundary first law from the engine's own report and requires it to close within one percent of the energy the plant exchanges. Plants that do not yet close are not hidden: they are *pinned* by name with their measured residual, and the pin ratchets in both directions. A pinned debt that grows fails; one that starts to close also fails, asking for its pin to be tightened, so the ledger always describes the engine as it is. The denominator matters and was argued, not chosen: a residual divided by the enthalpy throughput, most of which is the formation datum passing through, looks small when the plant is wrong; divided by the energy exchanged, it does not.

An unknown word is refused. Where a unit chose its sub-model through a chain of comparisons ending in a catch-all, a one-character slip in the model name silently selected the default — in one case with every published indicator byte-identical, so no golden master could ever see it. Every such site now refuses through one shared function that names the accepted words and the closest one, and the gate counts the sites rather than trusting a list.

A size is recomputed from the declaration. A specification sheet is generated for every sized item, and the gate recomputes sizes from what the case declared rather than from what the run printed. The class was found when a crystalliser had been costed on a volume computed from a molar flow — an order of magnitude off, at exit zero, pinned to four decimals by its own golden master. A golden master pins what a run prints; only a recomputation from the declaration can see a size that is wrong.

A further gate illustrates that the class need not be physical. `check_wasm_dialect` parses every source file with a second compiler that refuses the language extensions the WebAssembly toolchain refuses, because code that builds natively and not for the browser once left the public site serving a stale engine while the native suite was green. None of this makes the engine error-free. It makes each class of error, once seen, unable to return without a gate failing — and it makes the remaining open debts a list with names and numbers rather than a hope.

6 A comparison with an open-source simulator

TODO (authors): Publish this section only after the reference's defects listed below have been offered to its maintainers as an upstream issue (draft: `paper/whitepaper/upstream-issue-draft.md`). The benchmark cases are on branch `claude/c123-benchmarks-bc` (commit `33e277ca2`) and not yet on `main`; update the citation when they land.

We compared Choupo with DWSIM [7], an open-source process simulator licensed under the GPL-3.0, on samples from the library it ships. We name it because it is open: its licence and source could be read before anything was relied on, and its maintainers can reproduce everything below. Commercial simulators are not compared, because their models cannot be inspected.

Method. Each sample was rewritten as a Choupo case from its published problem — feeds, units, specifications — and no file or parameter of the sample was copied: the samples embed pair parameters from databanks this project excludes, so every property number in a benchmark case is Choupo's own or comes from a primary source. The reference numbers are read from the stored solved state inside the sample file, cited by the repository commit and by the build version the file records; every workaround is stated in the case.

DRAFT

Table 2: Case B: selected results against the reference's stored state (Choupo with its own regressed NRTL pair). The reference's own stored state closes its material balance to +0.14 % and its first law to +1.47 kW of about 4640 kW exchanged; Choupo closes every unit's mass balance and its plant first law to +0.079 kW.

	reference	**Choupo**	**difference**
LP condenser duty [kW]	2575.5	2609.9	+1.3 %
LP reboiler duty [kW]	2563.3	2600.8	+1.5 %
HP condenser duty [kW]	2054.7	2041.6	−0.6 %
HP reboiler duty [kW]	2076.5	2058.0	−0.9 %
LP distillate x_{MeOH}	0.2412	0.2243	−0.017
HP distillate x_{MeOH}	0.3066	0.2844	−0.022
acetone product T [K] (pure, 6 atm)	394.95	392.89	−2.05 K
methanol product T [K] (pure, 1 atm)	337.90	337.70	−0.20 K

Case B: methanol–acetone pressure-swing distillation. The sample (its file calls it extractive; it has no entrainer) couples a 40-stage column at 1 atm and a 20-stage column at 6 atm, with the high-pressure distillate recycled. The sample file records BuildVersion 8.8.0.0, saved 2024-07-09, read at repository commit `1abf72d1b`. Choupo ran it with an acetone–methanol NRTL pair it regressed itself from measured bubble temperatures — a pair whose held-out verdict was *not validated*, used case-locally and announced as such — and, as a second variant, with predictive UNIFAC.

The duties agree to between −0.9 and +1.5 % (Table 2). The differences are explained by the property models, and the case shows which: the acetone product leaves the high-pressure column as pure acetone, where no activity model is involved, and its temperature differs by 2.05 K — the difference between the two programs' vapour-pressure correlations at 6 atm, and nothing else. The distillate compositions follow from the material balances rather than from an azeotrope, since neither column's top is pinched at these specifications. We would not expect agreement to the last digit: two independent implementations with different vapour-pressure correlations and different pair evidence that agreed exactly would suggest one had been tuned to the other.

What the comparison found on each side. The reference's stored state does not close: its two bottoms specifications admit no steady state (two products of 0.5 mol/s from a feed of 0.9986 mol/s, +0.14 %), and the recycle stream it carries differs by 1.47 % from the stream it copies while the recycle block reports convergence after one iteration. These are facts about one stored file, measured on its own numbers; they bound how finely the comparison can resolve and are being offered to the maintainers. The comparison also found two issues on Choupo's side, and they are the more useful half. A multi-feed column accepted a feed on its condenser stage and then omitted it from that stage's balance — the default mass-balance report flagged the plant at 109 % closure, but the run exited normally when it should have refused; and the column's "total condenser" publishes the vapour in equilibrium with the reflux, which is an extra equilibrium stage rather than a total condenser. The first is being corrected in the engine; the second is being analysed against the column's formulation, and any change to it will be shown with every result it moves before anything is re-recorded.

TODO (authors): When the column fixes land on `main`, cite their commits and the gate that holds each; until then do not claim a gate exists.

Case C, an open item. The second sample, a four-stage counter-current extraction of acetone from water with methyl isobutyl ketone, is not yet reproduced. Choupo's predictive UNIFAC did not describe the water–MIBK miscibility gap well enough, so a liquid–liquid NRTL set was curated from the measured tie lines of Choi and Rhim [8]; with it, Choupo's liquid–liquid flash

DRAFT

still returned a single liquid on feeds its own stability test had found unstable, and its extractor published a non-converged cascade with a warning instead of refusing. Both are engine defects, and correcting them is scheduled work. The comparison will also have a limit even then: the reference's column is adiabatic and cools by 13 K, while Choupo's extractor is isothermal by construction, so flows and recoveries will be comparable and stage temperatures will not.

7 Working with an agent

Choupo is written for a workflow in which an agent prepares and a student decides.

The agent reads what Choupo writes for it. The repository carries an entry point for AI agents (`AGENTS.md`) and a library of short reference files under `docs/ai/`: the dictionary grammar, the case layout, the unit-operation catalogue, patterns, pitfalls, and a curation protocol whose first rule is that an agent never invents a number. `bin/llmctx` concatenates the library, or a core subset of it, into one context an agent can be handed.

Refusals tell the agent what to fix. An agent writing case files makes the same slips a student makes: a model name misspelt, a stream specified by three variables when two fix it, a tear stream not declared. Because Choupo refuses these by name, quoting the rule and the remedy, an agent can correct its own case in the next turn instead of reporting a plausible wrong answer. Listing 3 is what the first tutorial returns when its feed is given a vapour fraction beside its temperature and pressure: the run stops with a non-zero exit status, before any computation, and the message is written to be acted on. The refusal is the interface.

Listing 3: A refusal by name (the first tutorial with `vaporFraction 0.3;` added to its feed); exit status 2.

```
ERROR: stream state 'feed': OVER-SPECIFIED -- declares T = 370 K, P = 100000 Pa AND vaporFraction = 0.3
for a MIXTURE of 2 components (benzene, toluene).  A stream of known component flows is fixed by TWO
intensive variables (Duhem's theorem); for a mixture (T, P) already fix the vapour fraction, so a third
value beside them is redundant when it agrees with the (T, P) flash and a contradiction when it does not.
  Remedies: (1) delete the `vaporFraction` line and let (T, P) resolve the split -- an unpinned stream
MEANS its own equilibrium at (T, P, z), and the stream table and result JSON show the vapour fraction
that resolution gives; (2) if the QUALITY is the specification, the state is (P, q) or (T, q) and the
other variable must be SOLVED from it -- this engine does not do that today [...], so declare the T that
gives your q, or use a phaseChanger to produce the state.  A PURE component keeps `vaporFraction q;`: on
its saturation curve (T, P) do not fix the split.
```

Data and fits: the agent prepares, the student decides. When a model needs data the catalogue lacks, the agent can locate measurements in the open ThermoML archive, extract them into a dataset with its source, fit the model's parameters on one study and hold them out on another, against a tolerance band declared before the fit. The fit produces a dossier and a verdict — validated, not validated, not tested — and promotes nothing: promotion into the curated catalogue is a human act. The browser's explorer shows the verdict beside each record. An evidence view that draws every measured point against every candidate model, for a component or a binary pair, is being built: the student, not the agent, chooses the model, with the evidence on the screen.

TODO (authors): Update this paragraph when the C117 evidence view lands on `main`.

The engine is not the only authority in this arrangement, and should not be: a model fitted honestly can still be the wrong model. But every claim the agent makes about a number can be traced to a run that printed it, a record that sourced it, and a gate that would have failed had it not conserved.

DRAFT

8 Pedagogy: auditing a design is the competence

A simulation course has traditionally taught students to *set up* a simulation: choose a property method, specify a column, converge a recycle. When an agent can do the setting up, the competence that remains is *auditing*: reading a design and finding where it is wrong. That is exactly what a glass-box simulator exercises. A student who sees that a vapour pressure was extrapolated asks whether the answer depends on it; a student who sees a plant's first law miss by three percent looks for the unit that carries the gap; a student who sees a refusal learns what the model needed.

Choupo supports this in three ways. The source is meant to be read: small classes, explicit factories, hand-written numerics. The run log shows the computation at the level of detail a lecture would: the K-values, the Rachford–Rice iterations, the composition loop. And a set of *EduTools* — short, interactive pages in the browser, each built around one idea and running the real engine where a computation is involved — covers ten disciplines from thermodynamics and separations to process safety and economics; the site's generated catalogue lists 61 of them live. Each lesson page defines every symbol it uses and was drafted against the engine's source, cited by file and line; we note that the definitions are held by a gate, the citations are an authoring discipline and are not.

A design produced by an agent and audited by a student against a tool like this is, we suggest, a better exercise than a design typed by the student into a tool that does not explain itself. Whether it is in fact better is an empirical question for the classroom, and one we have not yet answered.

TODO (authors): Vítor: any classroom use to report (course, year, number of students, what was observed)? If none, keep the sentence above as it is.

9 Limitations

Nothing in this paper should be read as a claim of breadth. Choupo does not aim at parity with industrial simulators: its component catalogue is curated rather than encyclopaedic, and it has no equation-oriented solver; its models cover what its tutorials exercise.

- *A regression pass is not validation.* Most of the corpus is compared with answers Choupo itself recorded; a pass proves the answer has not moved, not that it is right. The validation subset anchored to published measurements is small, and only some of its claims are checked by machine rather than stated in prose.
- *Open first-law debts are pinned, not closed.* The energy gate holds named plants that do not yet close their first law within one percent; they are listed with their residuals rather than hidden, and they are open.
- *Some balances have not been measured.* The first law of the time-integrated batch reactor and dynamic CSTR has not been measured by the slice that closed it for their steady counterparts, and is claimed neither way.
- *A gate's reach is its scope.* The benchmark cases are outside the regression suite, and a gate sees only the cases it walks. The stage-1 feed of Section 6 was flagged by the default report but not refused, because no tutorial exercised it.
- *Known numerical limits* are documented beside the feature they bound: bubble-point distillation through an azeotrope is unstable and the simultaneous method is the documented escape; the liquid–liquid flash of Section 6 is being corrected.
- *The pedagogical claim is untested.* We have argued that auditing an agent's design is the competence a simulation course should teach; we have not measured whether students learn it better this way.

DRAFT

10 Availability

Choupo is free software under the GNU General Public License, version 3 or later. The source, the tutorials, the guides and the browser application are at `https://github.com/choupo-admin/choupo` and `https://www.choupo.org`; the latest release is `Choupo-2608`, and the default branch carries the development line. `CITATION.cff` in the repository gives the preferred citation. The guides are licensed under CC BY-SA 4.0.

TODO (authors): Zenodo DOI for this white paper and for the release it cites; arXiv identifier; confirm the repository URL.

Statement on the use of AI assistance

Substantial parts of Choupo's implementation, documentation and tutorial corpus, and the first draft of this paper, were produced with the assistance of AI coding agents (Anthropic's Claude Code), working under written briefs. The project is human-curated: every part of it has been reviewed, corrected and accepted by its maintainers, and the authors have checked this paper's claims against the software and answer for its content. Copyright in Choupo belongs to its named human contributors.

Acknowledgements and trademarks

TODO (authors): Acknowledgements (funding: none / institutional support, if any).

Choupo is a registered trademark of TalentGround Lda. OPENFOAM is a registered trade mark of OpenCFD Ltd; Choupo is not part of, affiliated with, endorsed by, or sponsored by OpenCFD Ltd or the OpenFOAM Foundation. DWSIM is named only as the open-source software compared in Section 6; no endorsement by its authors is implied.

TODO (authors): Verify the exact attribution wording OpenCFD Ltd requests for third-party mentions of OPENFOAM (not found on openfoam.com or openfoam.org on 2026-10-21); the line above is the conservative statement already in `NOTICE`.

DRAFT

References

[1] H. G. Weller, G. Tabor, H. Jasak, and C. Fureby. A tensorial approach to computational continuum mechanics using object-oriented techniques. *Computers in Physics*, 12(6):620–631, 1998. doi: 10.1063/1.168744.

[2] National Aeronautics and Space Administration. Standard for models and simulations. Technical Report NASA-STD-7009, NASA, 2008.

[3] T. J. Williams and R. E. Otto. A generalized chemical processing model for the investigation of computer control. *Transactions of the American Institute of Electrical Engineers, Part I: Communication and Electronics*, 79(5):458–473, 1960.

[4] Wolfgang Wagner, J. R. Cooper, A. Dittmann, J. Kijima, H.-J. Kretzschmar, A. Kruse, R. Mareš, K. Oguchi, H. Sato, I. Stöcker, O. Šifner, Y. Takaishi, I. Tanishita, J. Trübenbach, and Th. Willkommen. The IAPWS industrial formulation 1997 for the thermodynamic properties of water and steam. *Journal of Engineering for Gas Turbines and Power*, 122(1): 150–184, 2000. doi: 10.1115/1.483186.

[5] W. J. Hamer and Y.-C. Wu. Osmotic coefficients and mean activity coefficients of uni-univalent electrolytes in water at 25 °C. *Journal of Physical and Chemical Reference Data*, 1 (4):1047–1100, 1972.

[6] L. N. Plummer and E. Busenberg. The solubilities of calcite, aragonite and vaterite in CO_2–H_2O solutions between 0 and 90 °C, and an evaluation of the aqueous model for the system $CaCO_3$–CO_2–H_2O. *Geochimica et Cosmochimica Acta*, 46(6):1011–1040, 1982.

[7] Daniel Wagner Oliveira de Medeiros. DWSIM: an open-source chemical process simulator. https://github.com/DanWBR/dwsim, 2024. GPL-3.0 (README and COPYING read at commit 1abf72d1b6b41d3e9a8cc770d3cc4e8fc76e5766, 2026-07-17); sample files carrying BuildVersion 8.8.0.0.

[8] J. S. Choi and J. N. Rhim. Liquid–liquid equilibria for ternary systems with solvent – water – acetone. *Hwahak Konghak (Journal of the Korean Institute of Chemical Engineers)*, 24(3): 185–194, 1986.

DRAFT